DECAF20 Parse Tree Submission 3

Christopher Harris
CS 4308 Concepts of Programming Languages – Section 01
College of Computing and Software Engineering
Kennesaw State University
Marietta, Georgia
charr396@students.kennesaw.edu
November 10, 2026

I. Problem

The DECAF20 scanner and parser iterate through a given DECAF20 source file to collect each token, gathering key attributes such as token type and line number, and catching syntax errors based on the order in which those tokens appear. The current implementation validates the code's grammatical structure, ensuring things like declarations, expressions, statements, and more follow the syntax rules. However, it is missing a comprehensive visual representation of the parsing process. This is why a parse tree is beneficial. It shows each type of statement, declaration, expression, and other components in the order they are parsed. The tree gives a structure to everything, providing a clear representation of the program's overall organization, making it easier to understand the parsing process and follow along.

II. Purpose and Summary

The purpose of this deliverable was to expand the existing Python parser for the DECAF20 programming language by adding a parse tree to it. This involved creating a new class to represent parse nodes within each existing parsing function. This architectural addition gives the parser the functionality to visually represent its results instead of just reporting whether parsing succeeded or not.

The ParseTreeNode class was created for building and displaying the parse tree. Each node holds a parse label for the kind of program component it represents, source code line number, and a list of child nodes. The class includes functions for adding children and recursively printing the entire tree. As the parser processes the file, it creates nodes according to the program's different components, such as declarations and statements. This is displayed once parsing is complete through a saved hierarchical structure.

The Parser class remains the same, with specialized parsing methods for DECAF20 function declarations, calls, and other valid syntax. Each method was extended so that the parse tree nodes could be created and returned within them, which are added to their parent nodes. This approach incrementally builds the parse tree as the parsing is completed. For expression parsing, different expression types and relationships were added, with grouping in the parse tree determined by operator precedence. None of the token validation or syntax-error reporting was modified, keeping the parser's original functionality intact while the parse tree is constructed alongside it.

III. Test Input and Output Results

The screenshot in Fig. 1 demonstrates the executed parser output of one test file, whose DECAF20 source code is shown below. In the output, the parser was able to completely parse and build the corresponding parse tree for the program file.

```
int factorial(int n)
{
  if (n <=1 ) return 1;
  return n*factorial(n-1);
}

void main()
{
   int n;
   for (n = 1; n <= 15; n = n + 1)
      Print(
        "Factorial(", n , ") = ",
        factorial(n),
        "\n"
      );
}
```

IV. Conclusion

The final version of the parser provides a complete overview of the parsing process and program code structure through the newly incorporated parse tree, extended from the base scanner and parsing code functions for the DECAF20 language. A program file starts by being scanned for all its tokens and attributes, which are then accumulated into a list. Then, the tokens are parsed according to the order in which they appear in the program to validate the correctness of the syntax. Finally, during the parsing process, parse tree nodes are created for each function call, declaration, expression, and other components. They are connected to create a full parse tree showing the program's full hierarchical structure.

The final parse tree visualizes the full step-by-step process of the parser instead of relying on a single successful parsing message or error line. It provides an overview that is helpful for pinpointing where each parse came from, what it identified,

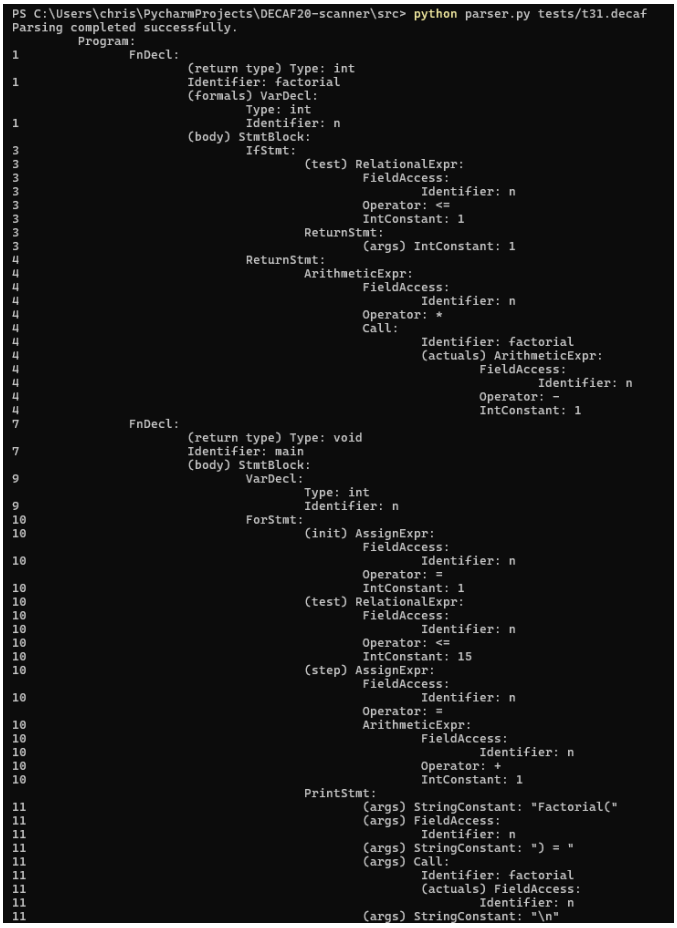


Fig. 1. Test Case Output

and its placement in the code. The final project deliverable closes out the DECAF20 language processing pipeline, showcasing the importance of lexical analysis, grammatical validation, and parse tree construction in analyzing, verifying, and representing source code syntax.

REFERENCES

[1] M. Alghoul, "Building a Custom Parser: A Step-by-Step Guide," *Medium*, Aug. 5, 2023. [Online]. Available: https://medium.com/@menta8seca/building-a-custom-parser-a-step-by-step-guide-b0a92340ff51

[2] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Pearson, 2019.